

DemocraSync System Pseudo-code

The following pseudo-code outlines the core logical flow of the DemocraSync application, including authentication, the AI OCR automated status assignment, and the secure voting mechanisms.

```
// =====
// MODULE 1: APP INITIALIZATION & ROUTING
// =====
FUNCTION Main():
    session = LocalStorage.get("userSession")

    IF session is EMPTY:
        NavigateTo(LoginScreen)
    ELSE IF session.role == "Admin":
        NavigateTo(AdminDashboard)
    ELSE IF session.role == "Student":
        NavigateTo(StudentDashboard)
END FUNCTION

// =====
// MODULE 2: REGISTRATION & AI OCR
// =====
FUNCTION RegisterStudent():
    studentData = GetInputFields(ID, Name, Course, Email, Password)
    corImage = FilePicker.pickImage()

    // AI OCR Check
    ocrResult = AI_OCR_Scanner.scan(corImage)

    IF ocrResult.isValid == TRUE:
        studentData.status = "Verified"
    ELSE:
        studentData.status = "Rejected"

    studentData.password = HashPassword(studentData.password)
    Database.collection("voters").save(studentData)

    ShowMessage("Registration Complete. Please Login.")
    NavigateTo(LoginScreen)
END FUNCTION

// =====
// MODULE 3: AUTHENTICATION
// =====
FUNCTION Login(userID, password):
    IF userID == "admin" AND password == "admin123":
        LocalStorage.save("userSession", {role: "Admin"})
        NavigateTo(AdminDashboard)
        RETURN

    userRecord = Database.collection("voters").find(userID)

    IF userRecord is NOT EMPTY AND CheckHash(password,
userRecord.password):
```

```

        LocalStorage.save("userSession", {role: "Student", id:
userID})
        NavigateTo(StudentDashboard)
    ELSE:
        ShowError("Invalid ID or Password")
END FUNCTION

// =====
// MODULE 4: STUDENT DASHBOARD & VOTING
// =====
FUNCTION OpenOfficialBallot(userID):
    userRecord = Database.collection("voters").find(userID)

    // The Status Gatekeeper
    IF userRecord.status != "Verified":
        ShowError("Access Denied: Your account status is " +
userRecord.status)
        RETURN TO StudentDashboard

    // If Verified, allow voting
    candidates = Database.collection("candidates").getAll()
    Display(BallotScreen, candidates)
END FUNCTION

FUNCTION CastVote(userID, selectedCandidates):
    encryptedVote = Encrypt(selectedCandidates)

    Database.collection("votes").save(encryptedVote)
    Database.collection("voters").update(userID, {hasVoted: TRUE})

    ShowMessage("Vote Successfully Cast!")
    GenerateVotingReceipt(userID, selectedCandidates)
    NavigateTo(StudentDashboard)
END FUNCTION

// =====
// MODULE 5: PROFILE SETTINGS & RE-UPLOAD
// =====
FUNCTION OpenProfileSettings(userID):
    userRecord = Database.collection("voters").find(userID)
    DisplayProfile(userRecord)

    IF userRecord.status == "Rejected":
        ShowButton("Update COR")

    ON CLICK "Update COR":
        newCorImage = FilePicker.pickImage()

        // Secondary AI OCR Check
        ocrResult = AI_OCR_Scanner.scan(newCorImage)

        IF ocrResult.isValid == TRUE:
            Database.collection("voters").update(userID, {
                corBase64: newCorImage,
                status: "Verified"
            })

```

```

        ShowMessage("Document Accepted. You are now
Verified.")
    ELSE:
        Database.collection("voters").update(userID, {
            corBase64: newCorImage,
            status: "Rejected"
        })
        ShowError("Document Invalid. Status remains
Rejected.")
END FUNCTION

// =====
// MODULE 6: ADMIN DASHBOARD
// =====
FUNCTION LoadAdminDashboard():
    activeElection =
Database.collection("settings").get("electionStatus")
    Display(LiveTallyBoard)

    ON CLICK "Toggle Election":
        activeElection = NOT activeElection
        Database.collection("settings").update({electionStatus:
activeElection})

    ON CLICK "Export Tally":
        tallyData = FetchAllVotes()
        GeneratePDF(tallyData)
END FUNCTION

```